

Building a Synthetic Dataset with LLMs

You need 500 labelled examples. You have none, no budget and no annotators. Here is how to generate them without poisoning your own results.



Why Generate Data At All

Four situations where synthetic data is the only practical option

COLD START

You are building an eval set for a system with no users yet. No logs, no real questions, no labels.

COVERAGE GAPS

Real data is skewed. 90% of logged questions are variations of three things. Edge cases and rare categories barely appear.

PRIVACY

Real customer data cannot leave your environment or go to contractors. Synthetic equivalents can.

SCALE

Generating a first draft and having humans **verify** is far cheaper than having them **author**.

🔗 THE HONEST FRAMING

Synthetic data is not a replacement for real data. It is a way to **bootstrap when you have none**, and to fill gaps real data will never cover on its own.



Three Uses, Three Quality Bars

What you are building it for changes how you build it

USE CASE	QUALITY BAR	WHAT MATTERS MOST
EVAL SET	HIGHEST every label human verified	Correctness of every label, a realistic difficulty spread, no contamination
FINE-TUNING	HIGH errors get baked into weights	Diversity above all, consistent output format, licence compliance
TESTING	MODERATE used to exercise code paths	Coverage of edge cases, volume over perfection, deterministic and reusable



An eval set is a measuring instrument. If it is wrong, every decision you make with it is wrong, and you will not know.



The Circularity Trap

Generating data with a model, then using it to judge that model

- 1 Model X generates the eval questions
- 2 Model X generates the "correct" answers
- 3 You evaluate Model X against them
- 4 Model X scores brilliantly, you call it excellent

! You measured whether the model **agrees with itself**. That number says nothing about real quality.

WHERE CIRCULARITY HIDES

- One model generates questions, answers **and** grades them
- Training data generated by the model you will compare against
- Eval questions phrased like your own system prompt
- A fine-tuned model evaluated on data its teacher wrote



The defence: separate the generator, the labeller and the judge. Different models, with a human somewhere in the loop.



AI Coach John

Follow



Repost

Ground It In Real Data

Seeded generation beats generation from nothing

× UNGROUNDED

Invents policies you do not have, uses generic wording, and nothing links it back to your docs, so you cannot verify the answer.

✓ GROUNDED BY A SEED

Every question is answerable, the source chunk is the label, and verification becomes mechanical.

The passage is given, so the answer is checkable

```
SEED_PROMPT = "" "Passage:
```

```
{passage}
```

Generate {n} questions a real user might ask, where this passage holds the complete answer.

- Answerable using ONLY this passage
- Vary phrasing: direct, conversational, indirect
- Never mention "the passage" in the question

Return JSON: [{{"question": ..., "answer": ...}]]""

```
def generate_from_chunk(chunk, n=3):
```

```
    out = llm.invoke(SEED_PROMPT.format(passage=chunk.text, n=n),
                    temperature=0.8)
```

```
    return [{*i, "source_chunk_id": chunk.id} for i in parse_json(out)]
```



The unlock: "correct" now means "retrieved the chunk this question came from", so retrieval labels cost you no human time at all.



AI Coach John

Follow



 Repost

The Diversity Problem

Ask for 500 examples and you get 40 ideas, repeated

❌ MODE COLLAPSE IN ONE CALL

```
"Generate 100 customer support questions"
```

1. How do I reset my password?
2. How can I reset my password?
3. What is the process to reset my password?
4. I forgot my password, what do I do?

Generation is not sampling a diverse space. It repeatedly takes the most likely path.

```
# Wrong: one call, high volume, low diversity
```

```
llm.invoke("Generate 100 customer support questions")
```

```
# Right: many small calls, varied conditioning
```

```
for persona in PERSONAS:
```

```
    for topic in TOPICS:
```

```
        for style in ["direct", "confused", "frustrated"]:
```

```
            llm.invoke(f"You are a {persona}. Write 2 questions "  
                        f"about {topic} in a {style} tone.",  
                        temperature=0.9)
```

```
# 8 personas x 12 topics x 4 styles x 2 = 768 examples
```



The principle: diversity comes from varying the input conditions, not from asking for variety. A model told to "be diverse" mostly is not.



Attribute-Driven Generation

Build a taxonomy first, then generate against it

```
from itertools import product
import random

TAXONOMY = {
    "topic": ["billing", "shipping", "returns", "account"],
    "intent": ["how_to", "troubleshoot", "policy_check"],
    "difficulty": ["simple_lookup", "multi_step", "inference"],
    "phrasing": ["direct", "conversational", "typo_ridden"],
    "persona": ["new_user", "power_user", "admin"],
}

def sample_combinations(taxonomy, n):
    # The full product is usually far too large, so sample it
    keys = list(taxonomy)
    combos = list(product(*(taxonomy[k] for k in keys)))
    return [dict(zip(keys, c)) for c in random.sample(combos, n)]
```

▣ GUARANTEED COVERAGE

Every intent by difficulty combination is represented on purpose.

🔍 MEASURABLE GAPS

"Only 4 inference examples on billing" is visible and fixable.

▣ STRATIFIED ANALYSIS

The strongest reason. "0.91 overall but 0.62 on multi_step" is a finding you can act on.

🔄 REPRODUCIBILITY

Regenerate the same distribution later when a topic is added.



Evolve Into Harder Examples

Generate easy, then deepen it systematically

🔗 ONE SEED, SIX VARIANTS

Seed question: **"What is the refund window?"**

CONSTRAINT

"What is the refund window for Pro plan customers in the EU?"

REASONING

"I bought on the 3rd and upgraded on the 10th, when does my window end?"

INDIRECT

"I bought this last month and it is not working, am I too late to send it back?"

AMBIGUITY

"How long do I have?" (a good system asks for clarification)

DISTRACTOR

"Thanks for the quick shipping by the way. What is the refund window?"

COMPRESS

"refund window" (typed the way people use a search box)



Two systems can both score 0.95 and be wildly different. The gap only shows on hard cases you never generated.



AI Coach John

Follow



Repost

Generating Hard Negatives

For retrieval and reranking, negatives matter more than positives

```
def mine_hard_negatives(question, positive_id, retriever, n=3):  
    # The best negatives are chunks that ALMOST answer it  
    hits = retriever.search(question, k=20)  
    return [c.id for c in hits if c.id != positive_id][:n]
```

```
HARD_NEG_PROMPT = """Write a passage that is topically very  
similar to this question and plausible, but does NOT answer it.  
Question: {question}  
Correct answer: {answer}  
It should be tempting to retrieve and genuinely wrong."""
```

1 **RANDOM** "Our office is open 9 to 5, Monday to Friday."

TEACHES NOTHING

2 **TOPICAL** "Returns must include the original packaging."

STILL EASY

3 **HARD** "Digital products are non-refundable once accessed."
(asked about physical goods)

REAL SIGNAL



If every wrong chunk in your corpus is obviously wrong, your retrieval metrics will look far better than production ever will.



AI Coach John

Follow



Repost

Most Of It Should Be Binned

Generation is cheap. Keeping bad examples is expensive.

```
for i, ex in enumerate(examples):
    q = ex["question"].strip()

    # 1. Structural validity
    if not 10 < len(q) < 500 or not ex.get("answer"):
        rejected["malformed"] += 1; continue

    # 2. Exact duplicates
    h = hashlib.sha256(q.lower().encode()).hexdigest()
    if h in seen: rejected["duplicate"] += 1; continue

    # 3. Near duplicates, semantic
    if any(np.dot(emb[i], emb[j]) > 0.95 for j in kept_idx):
        rejected["near_duplicate"] += 1; continue

    # 4. Generation artefacts
    if re.search(r"\b(the passage|the document|as an AI)\b", q, re.I):
        rejected["artefact"] += 1; continue

    # 5. Answer must be supported by the source
    if not is_grounded(ex["answer"], ex["source_text"]):
        rejected["ungrounded"] += 1; continue

    seen.add(h); kept.append(ex)
```



⚡ About 37% survival is normal and healthy. If you are keeping 95%, your filters are not filtering.



AI Coach John

Follow



Repost

Measuring Dataset Quality

Diversity and difficulty are measurable, so measure them

```
def dataset_diversity(questions, embedder) → dict:
    emb = embedder.encode(questions, normalize_embeddings=True)
    sims, n = emb @ emb.T, len(questions) # lower = diverse
    mean_sim = (sims.sum() - n) / (n * n - n)
    words = [w.lower() for q in questions for w in q.split()]
    return {
        "mean_pairwise_similarity": round(float(mean_sim), 3),
        "type_token_ratio": round(len(set(words)) / len(words), 3),
        "unique_openings": len({" ".join(q.split()[:3])
                                for q in questions}),
        "length_std": float(np.std([len(q.split())
                                    for q in questions])),
    }
```

SIMILARITY

High, above 0.7, means mode collapse. Vary the conditioning.

TOKEN RATIO

Low means the same vocabulary is being reused everywhere.

OPENINGS

Low means every question starts "How do I". Vary the phrasing.

LENGTH STD

Near zero means one shape. Add compress and expand variants.



Check the **distribution**, not just diversity. An eval set that is 62% easy lookups mostly measures easy lookups.



AI Coach John

Follow



Repost

How Much Verification

You do not need to check everything, but you must check something

EVAL SET

Verify 100% of labels. A wrong label makes every comparison subtly wrong. 50 to 200 examples is realistic and checkable in a day.

FINE-TUNING

Review a random 5 to 10%. If the error rate is above a few percent, fix the pipeline rather than the examples.

TESTING

Check edge and adversarial cases. Happy paths matter less, you are exercising code paths.

```
def sample_for_review(examples, n=50):  
    # Stratify so review covers the space, not just the bulk  
    by_stratum = defaultdict(list)  
    for ex in examples:  
        a = ex["attributes"]  
        by_stratum[(a["difficulty"], a["intent"])].append(ex)  
  
    per = max(1, n // len(by_stratum))  
    return [ex for g in by_stratum.values()  
            for ex in random.sample(g, min(per, len(g)))]
```

 **If 8% of reviewed examples are wrong, that is your dataset error rate. Record it and report scores with it in mind.**



Keeping Eval Data Uncontaminated

The failure mode that inflates every number you report

- 1 Train / eval overlap.** One seed chunk produced a training example and an eval example, so the answer was already seen.
- 2 Generator is the subject.** Questions written by Model X, then used to score Model X.
- 3 Generator is the judge.** One model wrote the reference answer and grades the output. Self preference inflates it.
- 4 Leaked phrasing.** Eval questions worded like your system prompt are unrealistically easy for your system alone.

```
def enforce_split(examples, test_fraction=0.2):  
    # Split by SOURCE DOCUMENT, never by example  
    doc_ids = sorted({ex["source_doc_id"] for ex in examples})  
    random.Random(42).shuffle(doc_ids)  
    test_docs = set(doc_ids[:int(len(doc_ids) * test_fraction)])  
  
    train = [e for e in examples if e["source_doc_id"] not in test_docs]  
    test = [e for e in examples if e["source_doc_id"] in test_docs]  
    return train, test
```



Two questions from the same paragraph are not independent examples.
Split by document, always.



Licensing & Terms Of Use

Check before you generate, not after you have trained

! THE ISSUE

Many commercial providers prohibit using their outputs to train, fine-tune or improve a competing model. Terms differ by provider and change over time.

Fine-tuning data generated from a commercial API

CHECK TERMS

Distillation from a commercial model into your own

CHECK TERMS

Eval sets used internally on your own system

USUALLY FINE

Test fixtures committed to your codebase

USUALLY FINE

```
{
  "generated_by":      "model-name-and-version",
  "generated_at":      "2026-08-15T10:22:00Z",
  "prompt_hash":       "a3f91e2b ... ",
  "seed_source":        "internal_docs_v4",
  "licence_note":       "internal evaluation use only",
  "human_verified":     true,
  "known_error_rate":  0.04
}
```



Record this in **every dataset file**, and ask legal before the dataset exists, not after it is inside shipped weights.



AI Coach John

Follow



Repost

Building A RAG Eval Set

End to end, the most common application

```
def build_rag_eval_set(chunks, n_per_chunk=2, target=200):
    examples = []
    # 1. Sample across the corpus, not just the first N chunks
    sampled = stratified_sample(chunks, by="source_doc_id",
                               n=target // n_per_chunk)
    # 2. Grounded questions, so the chunk IS the label
    for chunk in sampled:
        for ex in generate_from_chunk(chunk, n=n_per_chunk):
            examples.append(**ex, "expected_chunk_ids": [chunk.id],
                           "source_doc_id": chunk.doc_id)
    # 3. Evolve a third of them into harder variants
    for ex in random.sample(examples, len(examples) // 3):
        s = random.choice(list(EVOLUTION_STRATEGIES))
        examples.append(**ex, "question": evolve(ex["question"], s))
    # 4. Add questions the corpus CANNOT answer
    examples += generate_unanswerable(sampled, n=target // 10)
    return enforce_split(filter_dataset(examples, embedder)[0]) # 5
```

HIT RATE

Did we retrieve the expected chunk ids?

MRR

Was the right chunk near the top?

RECALL@K

Did we get all of the expected chunks?

REFUSAL RATE

Did we correctly say "I do not know" on unanswerables?



All four are **deterministic, free and instant**. Judge based answer quality comes later, and costs money.



AI Coach John

[Follow](#)[Repost](#)

Building Fine-Tuning Data

Different priorities: format consistency and diversity above all

```
def build_finetuning_set(seed_examples, target=2000):  
    # Self-instruct: expand 50 to 100 hand written seeds  
    pool = list(seed_examples)  
  
    while len(pool) < target:  
        # Few-shot from existing examples so style stays stable,  
        # but sample different ones so it does not collapse  
        demos = random.sample(pool, k=min(4, len(pool)))  
        new = llm.invoke(build_self_instruct_prompt(demos,  
                                                    taxonomy_sample()), temperature=0.9)  
  
        for c in parse_json(new):  
            if max_similarity(c["instruction"], pool, embedder) < 0.85:  
                pool.append(c)  
  
    return pool
```

FORMAT

Every example must follow the exact output shape you want learned. One inconsistent example is a taught exception.

DIVERSITY

500 diverse examples beat 5,000 near duplicates. The model generalises from variety.

DIFFICULTY

Include cases the model currently gets wrong, not only the ones it handles.

NOT VOLUME

Errors bake into weights permanently. Removing a bad example is cheap now and impossible after training.



Generating Test Fixtures

Generated once, committed to the repo, deterministic forever

```
# scripts/generate_fixtures.py, run ONCE and commit the output
def generate_test_fixtures():
    fixtures = {
        "happy_path": generate_cases("typical questions", n=10),
        "edge_cases": generate_cases("empty, long, emoji", n=10),
        "adversarial": generate_cases("prompt injection", n=10),
        "malformed": generate_cases("typos, fragments", n=10),
        "out_of_scope": generate_cases("unanswerable", n=10),
    }
    Path("tests/fixtures/cases.json").write_text(json.dumps(fixtures))
```

```
# tests/test_pipeline.py, the tests NEVER call an LLM
FIXTURES = json.loads(Path("tests/fixtures/cases.json").read_text())

@pytest.mark.parametrize("case", FIXTURES["adversarial"])
def test_resists_injection(case, pipeline):
    assert "system prompt" not in pipeline.run(case["input"]).lower()

@pytest.mark.parametrize("case", FIXTURES["out_of_scope"])
def test_refuses_unanswerable(case, pipeline):
    out = pipeline.run(case["input"]).lower()
    assert any(p in out for p in ["do not have", "cannot find"])
```



Generation happens once, in a script. Tests that generate at runtime are slow, costly, flaky and fail during provider outages.



Common Mistakes

What goes wrong when teams generate datasets

× GENERATION

- 500 examples in one call, 40 real ideas
- Ungrounded, inventing facts you do not have
- One model generates, answers and judges
- Random negatives instead of hard ones
- Every example the same difficulty

× HANDLING

- Splitting train and test by example
- Keeping 95%, filters that do not filter
- No human verification on an eval set
- No metadata on which model made what
- Tests that call an LLM at runtime

SYMPTOM	LIKELY CAUSE
"everything scores 0.95"	the dataset is all easy cases
"great in eval, poor in prod"	synthetic phrasing is not user phrasing
"fine-tune made it worse"	inconsistent format in training data
"eval score keeps rising"	train and test contamination
"reranker barely helps"	trained on random, not hard, negatives
"cannot reproduce last month"	no generation metadata recorded

The Trust Checklist

Run this before you trust a synthetic dataset

GENERATION

- ✓ Grounded in real source material
- ✓ Diversity from varied conditioning
- ✓ A taxonomy defines coverage first
- ✓ Difficulty gradient, not all easy
- ✓ Hard negatives, not random ones

INTEGRITY

- ✓ Generator, labeller, judge differ
- ✓ Split by source document
- ✓ No overlap of seeds and eval sources
- ✓ Provider terms checked

FILTERING

- ✓ Exact and near duplicates removed
- ✓ Generation artefacts stripped
- ✓ Answers verified against the source
- ✓ Survival rate recorded

VERIFICATION

- ✓ Eval labels 100% human verified
- ✓ Stratified sample reviewed
- ✓ Diversity measured, not assumed

PROVENANCE

- ✓ Model, version and date recorded
- ✓ Prompt hashed and stored
- ✓ Known error rate documented



Nineteen boxes. Any unticked box is a cheaper fix than regenerating the whole dataset later.



AI Coach John

Follow



PROTIBRIDGE
Solving For Better Futures

Repost

A Bootstrap, Not A Substitute

An LLM writes 2,000 examples in an hour. The value is in what you do next.

DO THIS	AND YOU GET
SEED FROM REAL DATA	every label is verifiable
VARY THE CONDITIONS	diversity you can measure
EVOLVE FOR DIFFICULTY	scores that discriminate
MINE HARD NEGATIVES	retrieval that actually learns
FILTER AGGRESSIVELY	about 40% survival, which is healthy
SEPARATE THE ROLES	no circular self agreement
SPLIT BY DOCUMENT	no contamination
VERIFY A SAMPLE	a known error rate
RECORD PROVENANCE	reproducible next quarter



Do this and you have a dataset you can decide with. Skip it and you have 2,000 confident looking rows that measure nothing. Follow AI Coach John for more.



AI Coach John

Follow



Repost